Autonomous Agent - Video Submission Script

Live Demo, Architecture, Debugging, and Tradeoffs

Assessment: Python AI Engineer - Autonomous Agents

Target runtime: 8-10 minutes (recommended run: ~9 minutes)

RUN OF SHOW

Target Criteria: Keep the Entire Recording Within 8-10 Minutes

0:00-3:30 — Live Demo: Run the standard request, show the downloaded document, then submit the ambiguous retreat request.

3:30-6:00 — What You Built: Walk through orchestration, planning, executor safety, rendering, and the API boundary.

6:00-7:30 — Debugging Insight: Explain the concrete context-auto-injection fix and prompt hardening.

7:30-9:00 — Tradeoffs and Close: Cover autonomy, local-model latency, reflection value, and finish with the readiness checklist.

Recording pace: Read the narration naturally, but do not read code line-by-line. Use each highlighted callout as a visual cue and keep file tours to the named function.

1. LIVE DEMO (3-4 min)

Target Criteria: End-to-End Execution & Two Test Inputs

SCREEN SETUP
Show React UI on localhost:5173 (left) and FastAPI logs on localhost:8000 (right).

ACTION 1 - STANDARD REQUEST
Paste the logistics proposal request into the UI and click Generate.

Narration: “First, a standard request. The UI shows a loading spinner. Look at the FastAPI terminal: the agent is making multiple calls to the local Ollama model. It first plans the TODO list, executes the tools, and finally renders the .docx. The important point is that this is not a single text-generation call: the request moves through a visible, auditable workflow. Keep this explanation to about thirty seconds while the model is running.”

```
Create a business proposal for a mid-sized logistics company adopting AI-based route
optimization...
```

ACTION 2 - SHOW OUTPUT
Show the completed UI summary. Click Download, open the .docx in Word, then show the tit
le page, TOC, and populated sections.

Narration: “The API response gives the frontend both a natural-language summary and a download URL. In Word, quickly show the title page, the updateable table of contents, and a few populated sections. Do not scroll through every page; this should take about thirty seconds.”

ACTION 3 - COMPLEX REQUEST
Paste the team-retreat project-plan request and click Generate.

Narration: “Now, the complex request. This tests missing information and conflicting requirements. Notice the Agent Assumptions box in the UI—the agent explicitly states it assumed a 3-day retreat and a tight-but-premium budget. This proves autonomous decision-making. It also makes uncertainty inspectable instead of silently inventing facts, which is important for business-document workflows. Spend no more than forty-five seconds on this example.”

```
We need a project plan for our team retreat next quarter. Budget is tight but we want it
to feel premium. Attendance is optional but management wants 90%. We are not sure if it is
2 or 3 days...
```

ACTION 4 - AGENT TASK LIST
Show the JSON response or terminal output and highlight plan.todos with every status set to done.

```
"plan": { "todos": [{ "tool": "generate_outline", "status": "done" }, ...] }
```

2. WHAT YOU BUILT (2-3 min)

Target Criteria: Architecture, Planning Logic, Tool Orchestration, API Design, Mandatory Improvement

FILE TO SHOW
app/agent/orchestrator.py - run_agent function

Narration: “The Orchestrator is the brain stem of the application. It is intentionally kept very small. It does four things: it asks the Planner to generate the plan, hands the plan to the Executor to run the tools, asks the Reflector to check for missing sections, and finally renders the document. We also capped the reflection to one follow-up pass, which prevents infinite loops if the model gets confused. Keeping this function thin matters because each component can be tested independently: orchestration controls sequence, while the planner, executor, and reflector each own one clear responsibility. Spend about thirty-five seconds here, then point to the next file.”

```
async def run_agent(request, ollama, request_id, session_id): plan = await plan(request,
ollama, context) state = await executor.execute(plan) reflection = await reflect(request,
plan, state.results, ollama)
```

FILE TO SHOW
app/agent/planner.py - plan function

Narration: “The Planner uses the qwen2.5-coder 7-billion parameter model. It outputs strict JSON. Pydantic validates the shape of this JSON. But we also added semantic repairs in Python: even if the LLM forgets, we force the plan to have an outline as step 1, and a document render as the final step. This is our Mandatory Engineering Improvement: Multi-step Planning. In other words, the model proposes the work, but deterministic Python code protects the workflow invariants that the rest of the system depends on. Keep this explanation to about thirty seconds.”

FILE TO SHOW
app/agent/executor.py - execute function

Narration: “The Executor runs the TODOs one by one using our Tool Registry. The most important feature here is argument sanitization. Small local models hallucinate arguments, so the executor strips out anything that isn't explicitly defined in the tool's schema. If a non-render tool fails, it records the error and keeps going so we still get a partial document. But if the renderer fails, it stops.”

```
This distinction is intentional: partial research or prose is still useful for review,
whereas a successful run without a downloadable document would not meet the product goal.
Cover this in roughly thirty-five seconds.
```

FILE TO SHOW
app/tools/docx_renderer.py - render_docx function

Narration: “The Renderer uses python-docx and OxmlElement to inject a real Word TOC field, page numbers, and styled tables, ensuring the final document is polished.”

FILE TO SHOW
app/main.py - post_agent route

Narration: “The API design is clean: a single POST /agent route. Notice that the Ollama client is injected as a dependency. This is a standard software engineering pattern that keeps our endpoint logic separate from the model client state. It also means our 31 automated tests can run completely offline using a fake Ollama mock. The route itself stays intentionally thin: it validates input, starts the agent flow, and shapes the response for the React frontend. This is why the API layer remains easy to reason about and test.”

3. DEBUGGING INSIGHT (1-2 min)

Target Criteria: Debugging Ability, Problem-Solving Approach

FILE TO SHOW
app/agent/executor.py - context auto-injection logic

Narration: “The hardest bug was that the 3B and 7B models kept failing to pass the correct arguments to the tools. For example, they would call generate_section_content with just {section_type: 'executive_summary'} but miss the required doc_title and data_block.

The root cause was that LLMs are bad at maintaining schema state. My fix was to build a 'Context Auto-Injection' pattern. The executor ignores whatever arguments the LLM passes for these specific tools and constructs the dictionary from scratch using the plan metadata.”

```
if todo.tool == "generate_section_content": args = {"doc_type": plan.doc_type.value,
"doc_title": plan.title, "section_title": section_title, "data_block": data_block,
"ollama": self._ollama}
```

FILE TO SHOW
app/llm/prompts.py - planner_system_prompt

Narration: “I also updated the prompt to explicitly ban hallucinated tools like add_table. The result is a resilient system that handles bad model outputs gracefully without crashing.”

4. TRADEOFF DISCUSSION (1-2 min)

Target Criteria: Scalability, Architecture Thinking

FILE TO SHOW
app/main.py - dependency injection

Narration: “Tradeoff 1: Autonomous Planning vs Deterministic Workflows. We chose autonomous planning—the LLM creates the TODO list. This is flexible but risky. We mitigated the risk with the Reflection stage and the executor's fallback logic. For a production system, this boundary is explicit: more reliability could come from tighter templates, but at the cost of flexibility. The current design makes that choice visible rather than hiding it: every generated plan is returned to the client for transparency.”

NARRATION
The second major tradeoff is Speed versus Functionality. We chose local Ollama models to avoid API costs and ensure data privacy. The tradeoff is that they are slow. A document

FILE TO SHOW
app/agent/reflector.py - reflect function

Narration: “Tradeoff 3: Multi-step planning adds latency because of the extra LLM calls for reflection. But it significantly improves coverage—the agent catches missing sections and retries them. The one-pass cap is the practical compromise: it improves quality without risking an expensive reflection loop. This gives the agent one chance to self-correct, while preserving a predictable upper bound on latency and model usage.”

5. FINAL CHECKLIST

Target Criteria: Recording Readiness

- Delete old output files before recording.
- Start uvicorn app.main:app --reload in Terminal 1.
- Start npm run dev in frontend (Terminal 2).
- Have Word open and ready.